Week 23
Safe Exploration, Shielding, CBFs, Robust RL

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives **3**

1 Motivation & intuition **4**
1.1 When constraints don't go far enough . . . 4
1.2 Robustness as a separate axis . . . 4

2 Mathematical formulation **5**
2.1 LTL primer . . . 5
2.2 Safety automaton and shield synthesis . . . 5
2.3 Safety layer (Dalal et al. 2018) . . . 6
2.4 Control Barrier Functions . . . 6
2.5 Robust RL: domain randomisation and adversarial training . . . 6

3 Algorithms **8**
3.1 Shielded Q-learning pseudocode . . . 8
3.2 Safety-layer pseudocode (continuous control) . . . 8
3.3 Domain randomisation training . . . 8

4 Implementation walkthrough **10**
4.1 The danger gridworld and its DFA . . . 10
4.2 Q-learning under the shield . . . 10
4.3 Pendulum with DR . . . 10
4.4 Robustness comparison . . . 11

5 Empirical results **12**

6 Exercises **14**

7 Further reading **15**

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Express a safety property in Linear Temporal Logic (LTL) and translate it into a deterministic safety automaton.
2. Synthesise a pre-shield from a safety automaton plus a one-step environment model; prove its soundness.
3. Implement an LTL-shielded Q-learning loop in which the agent never commits an unsafe transition during training.
4. State the safety-layer / differentiable-QP formulation (Dalal et al. 2018) and write down the closed-form projection for a single linear constraint.
5. Derive the Control Barrier Function (CBF) condition and the per-step CBF–QP filter; reason about forward invariance.
6. Compare domain randomisation, automatic domain randomisation (ADR), and adversarial training (RARL) as strategies for robust policy learning.
7. Empirically demonstrate the OOD-generalisation benefit of domain randomisation on a pendulum task.

1 Motivation & intuition

1.1 When constraints don't go far enough

Week 22's Lagrangian methods give us asymptotic constraint satisfaction — in the limit, $J_c(\pi^\star) \le d$. But the policy is allowed to violate the constraint along the way. For aerial vehicles, surgical robots, and re-ranking systems with hard fairness mandates, "asymptotic" isn't enough. We need to prevent unsafe transitions *during training, including the first episode.*

There are three classical families of techniques:

1. **Shielding** (Alshiekh et al. 2018, arXiv:1708.08611) — a formal-methods construction: filter actions against a finite-state *safety automaton* compiled from a temporal-logic spec.
2. **Safety layers** (Dalal et al. 2018, arXiv:1801.08757) — differentiable QP that projects the agent's action onto a half-space of safe actions.
3. **Control barrier functions** (CBFs) — a control-theoretic construction: define a function $h(s) \ge 0$ for the safe set and enforce a Lyapunov-style derivative condition.

The common thread: *decouple* the safety guarantee from the learning algorithm. The agent can be anything — Q-learning, PPO, SAC, a frozen LLM-policy — and the safety wrapper still works, with provable invariants.

1.2 Robustness as a separate axis

The second half of this week is *robust* RL: making the policy insensitive to perturbations of the environment dynamics. The two engineering moves are (i) **domain randomisation** (training on a distribution of environments broad enough to cover the deployment one) and (ii) **adversarial training** (a co-trained adversary perturbs the dynamics to make the protagonist robust). Both reduce to "don't overfit to one dynamics function".

Intuition. A shield resembles a railway interlock: a proposed move is checked against a model of which moves preserve a specified invariant. Its guarantee depends on that model and on the continued availability of a safe move. Cross-validation, by contrast, estimates generalisation; it does not enforce it.

2 Mathematical formulation

2.1 LTL primer

Definition 2.1.1 (LTL syntax)**.** Atomic propositions $p \in AP$ combine via Boolean connectives and the temporal operators $\mathbf{X}$ (next), $\mathbf{G}$ (globally), $\mathbf{F}$ (eventually), $\mathbf{U}$ (until). The grammar is

$$\varphi \;:=\; p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathbf{X}\varphi \mid \varphi\,\mathbf{U}\,\varphi.$$

$\mathbf{F}\varphi \triangleq \top\,\mathbf{U}\,\varphi$ and $\mathbf{G}\varphi \triangleq \neg\mathbf{F}\neg\varphi$.

The semantics is over infinite traces $w = w_0 w_1 \ldots$ of labels $w_i \subseteq AP$. Key fragment for safety: the *safety LTL* fragment

$$\mathbf{G}\,\psi(w_0, w_1, \ldots, w_k)$$

of *always*-properties on bounded windows. Every safety LTL formula admits a deterministic finite automaton (DFA) on traces; violation is reaching a designated rejecting sink.

Example 2.1.2 (The lecture's running formula)**.** $\varphi = \mathbf{G}\,\neg\text{danger}$ — "never enter a danger cell". The DFA has two states q_0 (accepting, "so far safe") and q_{bad} (rejecting, absorbing).

2.2 Safety automaton and shield synthesis

Definition 2.2.1 (Safety automaton)**.** A safety automaton for φ is $\mathcal{A}_\varphi = (Q, \Sigma, \delta, q_0, F)$ with $\Sigma = 2^{AP}$, $\delta : Q \times \Sigma \to Q$, initial q_0 and accepting set $F \subseteq Q$. An infinite trace is accepted iff every prefix lands in F.

Definition 2.2.2 (Pre-shield)**.** Given a one-step environment model $M : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ and a safety automaton $\mathcal{A}_\varphi$, the *pre-shield* maps each (s, q) to the safe action subset

$$\mathrm{safe}(s, q) \triangleq \{\, a \in \mathcal{A} \mid \delta(q, L(M(s, a))) \in F \,\},$$

where $L : \mathcal{S} \to 2^{AP}$ is the state-labelling function. The shielded policy is $\pi_{\text{shielded}}(\cdot|s, q) \propto \pi(\cdot|s) \cdot \mathbb{1}_{\mathrm{safe}(s,q)}$.

Theorem 2.2.3 (Soundness of the pre-shield)**.** *Suppose the environment is deterministic, M is exact, and* $\mathrm{safe}(s, q)$ *is non-empty for every reachable (s, q), the initial label is safe, and the shielded sampler can assign positive mass to a safe action. Then every trajectory of the shielded policy satisfies φ.*

Proof. By induction on t. Base: $q_0 \in F$ by construction. Step: at (s_t, q_t) with $q_t \in F$, the shield by definition 2.2.2 picks a_t such that $\delta(q_t, L(s_{t+1})) \in F$. So $q_{t+1} \in F$ and the trace prefix of length $t + 1$ is in F^*. Hence every prefix is in F^*, which by definition of the safety automaton means the trace satisfies φ. □

Remark. For stochastic dynamics a robust shield must check every possible successor and keep the next state in a controllable safe set. For learned M, additional coverage assumptions are required; soundness is not automatic: the shield is correct on the support of M, and mis-modelling drives a violation probability. Learning-based shields (arXiv:2410.07359, Oct 2024) bound this probability via concentration inequalities on the learned model.

Common pitfall

Empty safe set. If at some (s, q) the shield finds no safe action, the next step may be unavoidable even though no violation has happened yet: a one-step-safe move can enter a state from which every next move is unsafe. Standard recipes: (i) require an *invariant controller* that keeps the agent inside a known-controllable subset; (ii) expand the spec to include a recovery clause; (iii) fail-safe by stopping.

2.3 Safety layer (Dalal et al. 2018)

For continuous actions and a single linear constraint $g(s) + \nabla g(s)^\top a \le 0$, the safety layer solves

$$a^\star = \arg\min_a \|a - a_\pi\|^2 \quad \text{s.t.} \quad g(s) + \nabla g(s)^\top a \le 0.$$

For a single linear constraint the closed-form projection is

$$a^\star = a_\pi - \max\left(0, \frac{g(s) + \nabla g(s)^\top a_\pi}{\|\nabla g(s)\|^2}\right) \nabla g(s). \tag{2.1}$$

For multiple constraints, OptNet-style differentiable QPs (Amos & Kolter 2017) make $\partial a^\star / \partial \theta$ usable as a backward layer.

2.4 Control Barrier Functions

Definition 2.4.1 (CBF, control-affine systems)**.** For $\dot{s} = f(s) + g(s)a$, a continuously differentiable $h : \mathcal{S} \to \mathbb{R}$ is a (zeroing) CBF for the safe set $\mathcal{C} = \{s : h(s) \ge 0\}$ if there exists a class-$\mathcal{K}$ function α such that

$$\sup_{a \in \mathcal{A}} \left[\nabla h(s)^\top f(s) + \nabla h(s)^\top g(s)\, a \right] \;\ge\; -\alpha(h(s)).$$

Theorem 2.4.2 (Forward invariance)**.** *If $a(t)$ is any control law that for all t satisfies $\nabla h(s)^\top f(s) + \nabla h(s)^\top g(s)\, a \ge -\alpha(h(s))$, then $h(s(t)) \ge 0$ for all $t \ge 0$ provided $h(s(0)) \ge 0$.*

Proof sketch. $\dot{h} \ge -\alpha(h)$ along trajectories. Comparison principle for ODEs: $h(t) \ge y(t)$ where y solves $\dot{y} = -\alpha(y), y(0) = h(0) \ge 0$. Class-$\mathcal{K}$ α implies $y(t) \ge 0$ for all t, hence $h(t) \ge 0$. □

The runtime safety filter is a per-step QP:

$$a^\star(s) = \arg\min_a \|a - a_\pi(s)\|^2 \quad \text{s.t.} \quad \nabla h(s)^\top g(s) a \ge -\alpha(h(s)) - \nabla h(s)^\top f(s). \tag{2.2}$$

For a 1-D constraint this is again the closed-form projection of eq. (2.1), with $g(s) := \nabla h(s)^\top g(s)$ and $b := -\alpha(h(s)) - \nabla h(s)^\top f(s)$.

Remark. HJ-reachability and CBF are duals: HJ gives the largest CBF-like function analytically by solving an HJI PDE; CBF chooses a simpler analytic h and checks the inequality. arXiv:2204.12507 (2022, extended 2024) iteratively refines h using HJ.

2.5 Robust RL: domain randomisation and adversarial training

Definition 2.5.1 (Robust MDP)**.** A Robust MDP $(\mathcal{S}, \mathcal{A}, \mathcal{U}, r, \gamma)$ replaces the transition kernel by an *uncertainty set* $\mathcal{U}$. The robust value is

$$V^\pi_{\text{rob}}(s) = \inf_{P \in \mathcal{U}} \mathbb{E}_{\pi, P}\left[\sum_t \gamma^t r(s_t, a_t) \,\middle|\, s_0 = s \right].$$

For *rectangular* $\mathcal{U}$ (i.e. independent uncertainty for each (s, a)) the robust Bellman operator T_{rob} is a γ-contraction, so robust value iteration converges (Iyengar 2005; Nilim & El Ghaoui 2005).

Domain randomisation (DR; Tobin et al. 2017, arXiv:1703.06907) side-steps the inner $\inf_P$ by sampling $P \sim \mathcal{D}$ during training and maximising the *average* return. This is robust-on-average rather than robust-worst-case, but in practice frequently outperforms more careful robust algorithms for sim-to-real.

ADR (Automatic Domain Randomisation; OpenAI 2019, arXiv:1910.07113) expands $\mathcal{D}$'s support whenever the agent's performance on the boundary parameters becomes good — a curriculum on randomness.

RARL (Pinto et al. 2017) plays a zero-sum game between a protagonist policy and a disturbance policy that perturbs the dynamics. At a Nash equilibrium the protagonist is robust to bounded adversarial perturbations.

3 Algorithms

3.1 Shielded Q-learning pseudocode

Algorithm: Shielded Q-learning (one episode)

Inputs: env $\mathcal{M}$, agent Q, shield with predicate unsafe(s, a).

1. Reset env; $s \leftarrow s_0$.
2. Until done:
   a) Build ϵ-greedy preference list σ over actions.
   b) Filter: $a \leftarrow$ first $a \in \sigma$ with $\neg$unsafe(s, a).
   c) Step env: $(s', r, \text{done}) = \mathcal{M}.\text{step}(a)$.
   d) Update: $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$.
   e) $s \leftarrow s'$.

Invariant: the trace never visits a danger cell (definition 2.2.3).

3.2 Safety-layer pseudocode (continuous control)

Algorithm: Safety layer wrapper

1. Forward through actor: $a_\pi = \pi_\theta(s)$.
2. Compute $g(s)$ (constraint value) and $\nabla g(s)$ (its gradient).
3. Project: $a = a_\pi - \max(0, (g + \nabla g^\top a_\pi)/\|\nabla g\|^2)\nabla g$.
4. Step env with a; record (s, a, r, s'); backprop through the projection.

3.3 Domain randomisation training

Algorithm: DR-PG (one epoch)

1. Sample T rollouts; for each rollout, sample env parameters $\eta \sim \mathcal{D}$ at reset (e.g. pole length).
2. Compute Monte-Carlo returns and advantages with a value baseline.
3. PG step: $\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}_{\eta, \pi_\theta}[\log \pi_\theta A]$.

Common pitfall

DR plateau. If $\mathcal{D}$ is too broad, the gradient signal becomes the average of many incompatible policies — the agent learns nothing. Symptom: training return barely improves. Fix: start narrow, widen via ADR once the easy region is solved.

4 Implementation walkthrough

4.1 The danger gridworld and its DFA

```python
@dataclass
class DangerGridWorld:
    rows: int = 5; cols: int = 5
    start = (0, 0); goal = (4, 4)
    danger = ((1,2),(2,1),(2,2),(2,3),(3,2))
    def label(self, s_idx):
        return {"danger"} if self.idx2s(s_idx) in self.danger_set else set()
    def adjacent_dangerous(self, s_idx, a):
        # one-step lookahead - deterministic, so this is the complete model
        ...
```

The DFA has two states (fig. 5.1). The shield in `shield.py` implements the action filter: it iterates over preferred actions (ordered by Q-value, or by epsilon-greedy randomisation), returns the first action whose one-step image stays in q_0, and counts the number of overrides.

4.2 Q-learning under the shield

```python
def act(self, s, ep, shield=None):
    if rng.random() < eps(ep):
        order = list(rng.permutation(self.n_actions))
    else:
        order = list(np.argsort(-self.Q[s]))
    if shield is not None:
        return shield.filter(s, order)
    return order[0]
```

Crucial design choice: the shield is applied *inside the epsilon-greedy sampler*, not after. This means exploration is also shield-compliant; the ϵ-greedy random action is replaced by a random *safe* action.

4.3 Pendulum with DR

```python
@dataclass
class PendulumDR:
    randomise: bool = True
    length_low: float = 0.5; length_high: float = 1.5
    def reset(self, seed=None):
        ...
        if self.randomise:
            self.length = self.rng.uniform(self.length_low, self.length_high)
```

The state observation is $[\cos\theta, \sin\theta, \dot{\theta}]$ — no length channel — so the policy must implicitly handle the dynamics variation.

4.4 Robustness comparison

The training loop runs REINFORCE with a value baseline (no policy distribution information about the length is given). At test time we evaluate at fixed lengths $\{0.3, 0.6, 1.0, 1.4, 1.8\}$. The DR policy was trained on $[0.5, 1.5]$ so 0.3 and 1.8 are mild OOD points; the fixed policy was trained at 1.0 so all but the centre are OOD.

5 Empirical results

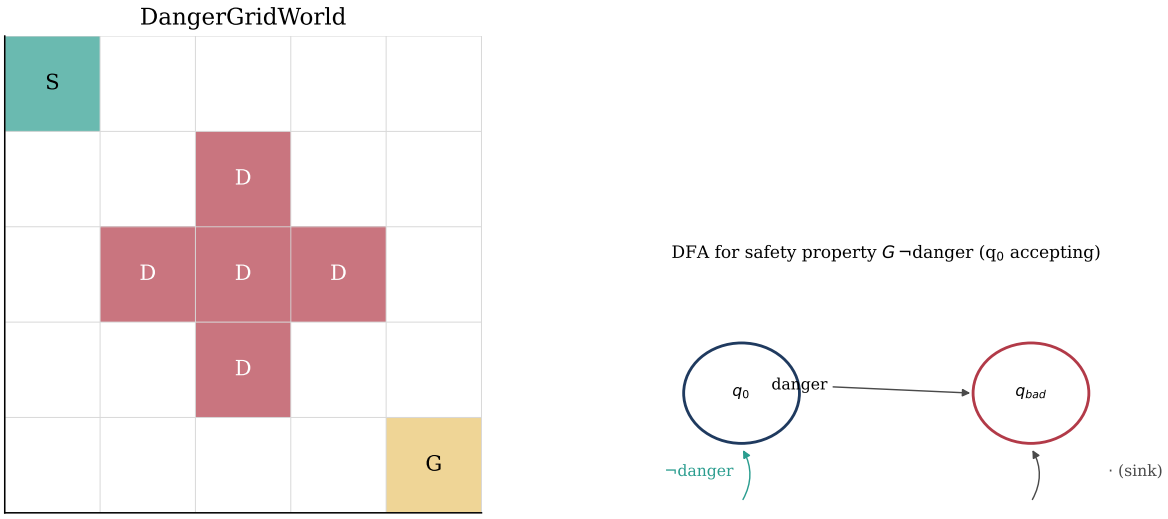


Figure 5.1: Left: the danger gridworld used for the shielded Q-learning demo. Right: the two-state safety automaton for $\mathbf{G}\,\neg$danger.

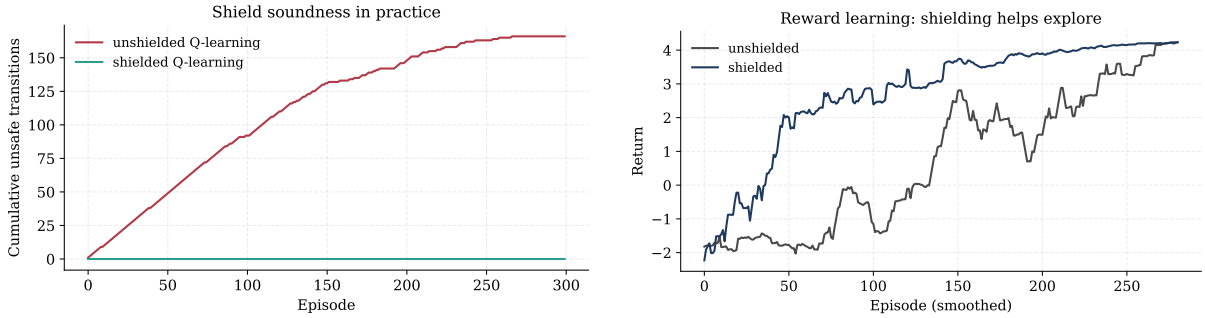


Figure 5.2: Left: cumulative unsafe transitions. The shielded curve sits at zero throughout (definition 2.2.3). Right: smoothed episodic return. The shielded agent learns at least as fast as the unshielded baseline — the shield's overrides serve as "free" exploration into unsafe regions without actually visiting them.

Safety angle

This is, ultimately, the chapter about *deployment*. The lecture's running message:

- **Shielding handles known-knowns.** If you have written down what "safe" means, you can synthesise a shield. If you haven't, no shield will save you.
- **Robust training handles known-unknowns.** DR makes the policy less brittle to dynamics you can enumerate (mass, friction, sensor noise). It does nothing for unknown-unknowns.

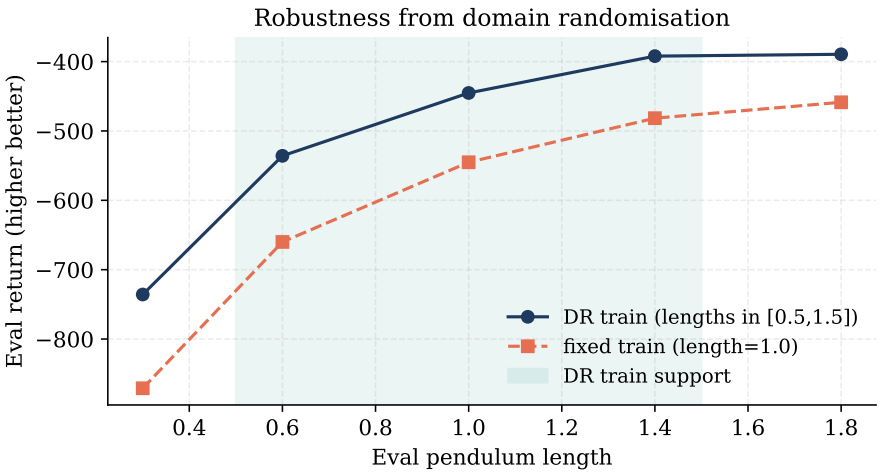


Figure 5.3: DR vs. fixed-length training, evaluated at a range of test lengths. Shaded region is the DR training support. The fixed-length policy collapses on OOD lengths; the DR policy degrades gracefully even outside its training distribution.

- **Composition matters.** In production we typically stack: offline-pretrained policy + Lagrangian-fine-tuned policy + CBF runtime filter + monitoring. Each layer addresses a different failure mode.

References: Alshiekh et al. 2018, arXiv:1708.08611, "Safe RL via Shielding"; Dalal et al. 2018, arXiv:1801.08757, "Safe exploration in continuous action spaces"; Tobin et al. 2017, arXiv:1703.06907, "Domain randomization for sim-to-real transfer". The 2024-2025 update is arXiv:2410.07359 on learning-based shielding under unknown dynamics, and arXiv:2502.13187 for foundation-model-conditioned sim-to-real.

Matching application

For synthetic ATS candidate-matching: a shield can enforce hard regulatory constraints (e.g. "never present a candidate whose audit flag is RED"), operating as a post-filter on the ranking policy's top-K output. The "state" is the current shortlist plus the candidate under consideration; the safety automaton is the verbalised hiring policy compiled into a checker. For synthetic insurance lead-matching, a CBF-style filter caps a single broker's daily allocation: $h(s) = q_{max} - n_{assigned}(s) \geq 0$. The runtime QP enforces the cap in real time, no matter what the underlying ranking model wants.
Domain randomisation generalises to *population shift*: train the re-ranker on a randomised mix of recruiter-quality profiles, candidate-supply shapes, and seasonal cohorts. The resulting policy is robust to month-to-month distribution drift in the live system without retraining.

6 Exercises

See `workbook.ipynb` for graded exercises with hints:

- **Ex. 1 ($\star$):** build the DFA for $\mathbf{G}\,\neg$danger.
- **Ex. 2 ($\star$):** unsafe-predicate one-step look-ahead.
- **Ex. 3 ($\star\star$):** implement the pre-shield filter.
- **Ex. 4 ($\star\star$):** shielded vs unshielded Q-learning.
- **Ex. 5 ($\star\star$):** verify shield soundness empirically.
- **Ex. 6 ($\star\star$):** DR vs fixed-length pendulum (meaningful objective).
- **Ex. 7 ($\star\star\star$):** 1-D CBF projection in closed form.
- **Ex. 8 ($\star\star\star$):** adversarial-extreme OOD evaluation.

7 Further reading

See `extended_reading.pdf`. Sourced from `research/safety_rl_sota.md` sections 2, 4, 9.